HYT AI Screening Agent: Report

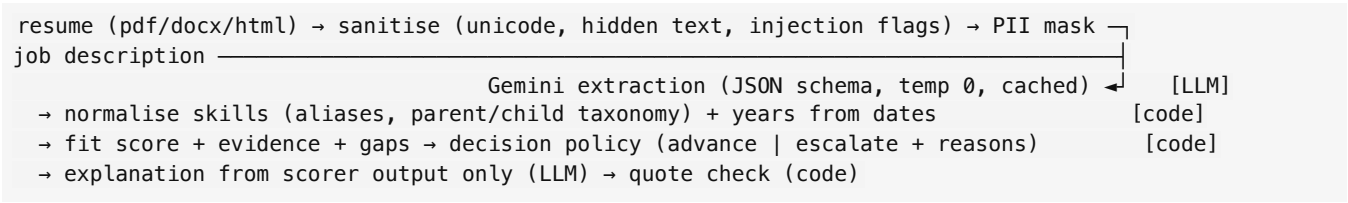
Job family: Software Engineering / IT. Corpus: 100 templated `.docx` resumes, plus 10 hand-picked UK JDs from `postings2.csv`. Golden set: 200 labelled (JD, resume) pairs, split dev 60 / test 140. All test numbers come from a system frozen on dev. Sources: `results/*.json`. Two demo apps wrap the same `agent.screen()` (README).

> **Reproduce:** `make reproduce-v1.0` regenerates every headline figure below (exact match, ~1 min, offline, no API key); `make reproduce` builds the current v1.1 system and all suites (~1.5 min).
>
> **Versions.** Headline numbers are the **frozen v1.0** system (archived in `results/v1_0_frozen/`). Two changes were made *after* the test evaluation, on evidence from real uploaded resumes and dev data. They are reported separately as **post-hoc v1.1** in §5b and are never mixed into the headline.
>
> **Status.** All suites have run. The **blind** fairness mode is missing (0/690 extractions, free-tier Gemini quota), so only the unblinded mode is reported. The LLM judge covers all 40 explanations, but agreement is currently measured against AI-made reference ratings (§6).

1. Problem framing

For a recruiter, "good" means two things. First, a **short list they can trust**: anything auto-shortlisted really meets the must-haves. Second, **nothing silently lost**: everyone else gets a reason and a queue position, not a rejection. So the agent only **auto-advances** or **escalates** (with reason codes and a low-priority queue for `LOW_FIT`). Missing a good candidate is weighted 5× worse than advancing a weak one. Not built: auto-reject, fine-tuning, multiple job families, and "ideal resume" vector retrieval (DECISIONS #2).

2. System



LLM reads, code judges. The fit score is `0.50·must + 0.15·nice + 0.20·experience + 0.15·role_relevance`. Components that are missing are renormalised away. Evidence strength is used = 1, listed = 0.5, learning = 0.25. "Used" must be visible in a role or project bullet, and code checks this. Children satisfy parents (Spring Boot ⇒ Java), never the reverse. Alternatives are satisfied by any one. Title technologies weigh ×2. A missing must-have multiplies fit by 0.6.

Auto-advance requires all of the following: fit ≥ T_high, every must-have ≥ 0.5, an explicit minimum-years requirement met, and no blocking flag (manipulation, extraction failure, low-confidence dates, sparse resume, non-English, overqualified, career switch).

3. Golden set

- **Sampling**: resumes split once 30/70 (seed 13). Each JD draws 20 from its split: BM25 top-6, BM25 middle-6, 4 random, 4 rule-picked hard cases. That gives 3 dev JDs and 7 test JDs, and no resume appears in both splits (validated).
- **Labels**: relevance 0–3 plus ideal decision. Written blind to scores and before the scorer existed, by Claude at the user's request (`golden/rubric.md`). Test distribution 0: 40 · 1: 64 · 2: 22 · 3: 14. Only 4 of 10 JDs have any candidate meeting every must-have.
- **Agreement**: an independent second rater (`gemini-3-flash-preview` with the same rubric, 30 pairs) gave **weighted κ = 0.98**, 97% exact relevance agreement and 100% decision agreement. Read this as "the rubric is applied consistently", not as "the labels are right": the rubric is nearly mechanical for templated resumes (§10).

4. Results vs baselines (test, 140 pairs, 7 JDs; stratified bootstrap 95% CI)

system	NDCG@5	NDCG@10	P@5	MRR	Spearman	auto-adv precision	recall (ideal adv.)	escalation	cost
pipeline	**0.98** [0.89, 1.00]	**0.91** [0.84, 0.96]	**0.60** [0.43, 0.71]	**1.00** [0.89, 1.00]	**0.65**	**1.00** (10/10)	0.43	0.93	**65**
LLM zero-shot (same model)	0.71 [0.66, 0.84]	0.79 [0.75, 0.87]	0.57	0.79	0.60	0.22	0.96	0.30	81
TF-IDF cosine	0.72 [0.63, 0.86]	0.74	0.46	0.80	0.21	0.17	0.96	0.08	112
keyword Jaccard	0.67 [0.58, 0.84]	0.74	0.43	0.71	0.04	0.18	0.87	0.22	104
embedding cosine	0.65 [0.56, 0.82]	0.70	0.43	0.70	0.16	0.16	1.00	0.00	117

The lexical baselines can't tell "meets the must-haves" from "is another software resume". The **zero-shot LLM ranks nearly as well by Spearman (0.60 vs 0.65), but decides badly**. Using its own advance/escalate judgement, it advanced 72 test candidates and **52 of them were wrong** (precision 0.28). The ranking signal is in the model; the trustworthy decision is in the code.

Extraction: schema-valid rate 1.00 (106/106, first attempt). Resume skill F1 1.00, YOE MAE 0.00 and education level 1.00 are trivially high on a template corpus (§10). **JD must-have F1 0.93, nice-to-have F1 0.67**, with JD01 at must-have F1 0.57. **Run-to-run** (3 fresh extraction runs of every test resume and JD, 207 new calls): score std **0.000**, decision flips **0%**, rank Spearman between runs **1.0**.

5. Decision policy

T_high was tuned on dev only (cost FN = 5, FP = 1). Dev has a zero-cost plateau from 0.40 to 0.49, and the highest value, **0.49**, was frozen. On test: **precision 1.00, recall 0.43, escalation 0.93** (34 to review, 96 low-priority). 117 of 130 escalations were correct. Reason codes: `MISSING_MUST_HAVE` 125, `LOW_FIT` 96, `LOCATION_MISMATCH` 36 (info only), `BELOW_THRESHOLD` 29, `CAREER_SWITCH` 22, `OVERQUALIFIED` 4.

A diagnostic ablation (added after seeing test errors; not used to change the frozen system) separates the 13 missed advances:

variant (test)	recall	precision	escalation	cost
frozen system	0.43	1.00	0.93	65
oracle JD requirements	0.78	1.00	0.87	25
`CAREER_SWITCH` non-blocking	0.61	1.00	0.90	45
both	**0.96**	**1.00**	0.84	**5**

Most of the loss comes from LLM requirement extraction, not scoring. **Product response**: the web workspace opens each job with an editable requirements panel. In the live app, a recruiter switching JD01's two wrongly extracted must-haves to "nice" shortlists **all 8 of 8** golden "should advance" candidates, with no LLM call.

5b. Post-hoc v1.1 (made after the test evaluation; measured one change at a time)

test, 140 pairs	Spearman	NDCG@10	recall	precision	escalation	cost
v1.0 frozen (headline)	0.648	0.912	0.43	1.00 (10/10)	0.93	65
+ framework categories in the skill map (DECISIONS #15a)	0.646	0.912	0.43	1.00	0.93	65
v1.1 = + `CAREER_SWITCH` informational (#15b)	0.646	0.912	**0.61**	**1.00 (14/14)**	0.90	**45**

- **#15a** came from a real uploaded resume: Express.js and FastAPI got no credit for "web frameworks such as Django or Flask". It changes 5 test scores and 0 decisions.

- **#15b** follows a dev-only tuning pass that could not calibrate the flag, because the corpus has no career switchers. Exactly 4 test decisions change (JD06/R055, JD06/R094, JD10/R094, JD10/R028), all escalate → advance and all labelled "advance". Dev results are identical for every variant (cost 0).
- **Security trade-off of #15b:** `CAREER_SWITCH` had been (accidentally) stopping 2 of the 4 "fake skills added to the skills list" attacks, because the attacked resumes' job titles didn't fit the role. Under v1.1 that attack auto-advances **4/4** (v1.0: 2/4); every other attack is unchanged (§8). A recall gain on honest candidates is paid for with weaker protection against a candidate who simply lies. The planned mitigation is to require at least one "used" (not just listed) must-have before auto-advance.
- **Caveat:** these changes were chosen with knowledge of the test errors (the ablation in §5 had already exposed `CAREER_SWITCH`), so v1.1's test numbers are **optimistic**. Only a fresh labelled set can confirm them. v1.1 is still far from the oracle ceiling (recall 0.96), because JD requirement extraction remains the main loss.

6. Explanation quality

- **Code groundedness: 1.00.** All 314 quoted spans in 140 test explanations appear in the visible resume (114/114 on dev).
- **Reference rating** (40 test explanations, rated before any judge output). **These ratings were made by Claude (the AI coding assistant), not by a human.** They must be replaced by the author's own ratings before judge–human agreement can be claimed (`golden/explanation_rating_sheet.md`): groundedness 4.1, gap correctness 4.7, specificity 4.0, usefulness 3.0. Quotes are safe, but claims are not always: 8 of 40 misread a reason code (`CAREER_SWITCH` as "a history of changing careers"), gave a wrong cause, or got the experience-gap arithmetic wrong. One (JD10/R004) said a degree was missing when it wasn't.
- **LLM judge: Claude Sonnet**, a different model family from the Gemini generator, run as a separate agent at low effort, blind to the reference ratings, with a fixed rubric (`eval/judge_prompt_sonnet.md`). It scored all 40 (`results/judge_scores.csv`): groundedness 4.2, gap correctness 4.4, specificity 3.3, usefulness 2.8, with 9/40 at groundedness ≤ 3. It independently caught the same real errors as the reference ratings: the false "missing degree" (JD10/R004 → 1), "0.5 years" read as "0.5 years short" (JD09/R050, R075), `CAREER_SWITCH` read as a career history, and invented causes of escalation.

judge vs reference, n = 40	groundedness	gap correctness	specificity	usefulness
weighted κ (quadratic)	0.66	0.53	0.02	0.41
Spearman	0.55	0.35	0.10	0.46
mean (reference / judge)	4.1 / 4.2	4.7 / 4.4	4.0 / 3.3	3.0 / 2.8

The judge is trustworthy for **groundedness** (it finds factual errors) and **usefulness**, and not for **specificity**, where the two raters disagree on what "specific" means. **Caveat:** the reference rater is Claude Opus, the same family as the judge, so these κ are likely optimistic. They become judge–*human* agreement only once the author's ratings replace `golden/explanation_human_ratings.csv` and `make judge` is re-run. A Gemini judge was tried first and abandoned: the free tier allowed only 7 of 40 items, and it gave 5/5 to all of them. * **Stale explanations**: an explanation describes one scoring. The workspace marks it *outdated* when requirements or the threshold change, instead of showing text that contradicts the new decision.

7. Fairness (counterfactual twins, unblinded mode)

30 test resumes, 13 single-signal comparisons, **657/690 variants extracted** (quota failures are excluded, never scored as empty). The blind mitigation mode got 0/690 extractions (quota) and is not reported.

- **Name** (male/female; Western vs South-Indian, North-Indian, Muslim, East-Asian), **pronouns**, **women-in-tech membership**, **elite vs lesser-known university**, **women's vs co-ed college**, **graduation year −10**, **metro vs small town**: **Δscore exactly 0 for every resume, 0 decision flips.** Identity signals never reach the score (name, institution and location aren't inputs), and extraction of the scored fields didn't change.
- **Writing fluency** (plain non-native English): mean |Δ| 0.003, max 0.010, 0 flips. The change is consistent (Wilcoxon $p < 0.001$) but tiny. It comes via role-relevance embeddings, as predicted in the plan.

- **Caregiving gap**: the suite **found a bug**. The extractor recorded "Career break – family caregiving (2016–2017)" as a job, adding a year of experience. The fix is in code (`experience.py` skips non-employment entries; DECISIONS #3). After it: mean |Δ| 0.004, p = 0.14, 0 flips. All 200 golden scores were unchanged.
- **Noise floor**: the fairness reruns were lost to quota. The robustness run (§4) shows zero run-to-run variance at temperature 0, so any non-zero Δ is attributable to the variant.

8. Adversarial robustness (4 real resumes × 8 attacks)

attack	detected	auto-advanced	mean Δscore
visible injection, hidden CSS, white 1px text, zero-width obfuscation, white 2pt `.docx` run	**100%**	0%	0.00
40-term keyword stuffing	**100%**	0%	+0.33
fake must-have skills added to the skills list	0%	**50%** (v1.1: **100%**)	+0.32
"reviewer comment" injection with no trigger words	0%	**25%**	+0.09

Overall detection is 75% and auto-advance after attack is 9% (3/32). Every hidden-text and trigger-phrase attack is stripped or caught. Two gaps remain. **Lying in the skills list can't be detected from text**: it earns listed-level (0.5) credit, which is enough when a JD's must-haves are few. And the **trigger-free injection** was read by the extractor as the candidate's skills. Mitigations for next week: extract skills only from defined resume sections, and require at least one "used" must-have before auto-advance.

9. Failure analysis: top 5 (real examples, hypotheses)

1. **JD responsibilities extracted as must-haves.** *Example:* JD01's "orchestrating build pipelines… integrating monitoring" became must-haves `ci/cd` and `monitoring`; all 8 good Java/Spring Boot candidates were capped and escalated. *Hypothesis:* the prompt treats core responsibilities as requirements when a JD has no explicit required/desirable split, and JD01 has none. *Evidence:* oracle requirements lift recall 0.43 → 0.78 (§5). *Response:* recruiter confirmation in the requirements panel.
2. **Skill-map gaps between category phrases and concrete skills.** *Example:* "web development frameworks such as Django or Flask" was cleaned to `web`, so an uploaded resume's Express.js and FastAPI got no credit. *Hypothesis:* JDs name categories while resumes name tools, and the cleaner strips the category words. *Response:* framework categories (#15a) plus a panel flag for unmapped requirements.
3. **`CAREER_SWITCH` misfires.** *Example:* JD06/R055 and R094, JD10/R094 and R028: genuine software engineers escalated as career switchers. *Hypothesis:* role relevance is an embedding cosine of short titles plus identical template bullets, which isn't comparable across jobs (dev JD08 median 0.12 vs JD07 0.55). Dev tuning was flat (#15b).
4. **Explanations misstate codes and numbers.** *Example:* JD10/R004 says a CS degree is missing (strength 1.0); JD09/R050 calls 0.5 years of experience "0.5 years below" a 4-year bar. 9/40 score ≤ 3 on groundedness by the judge (8/40 reference). *Hypothesis:* the payload gives reason codes without definitions and leaves gap arithmetic to the LLM. *Fix:* deterministic templates for codes and gaps.
5. **Lies and trigger-free injection get through.** *Example:* JD01/R047 with JD must-haves added to its skills list was auto-advanced (fake skills: 2/4 in v1.0, 4/4 in v1.1); a "reviewer comment" with no trigger words got 1/4 through. *Hypothesis:* listed-skill credit can't tell true from false, and the extractor reads skills from any text block. *Fix:* require ≥ 1 "used" must-have; extract skills from resume sections only.

Also observed: identical skill sets tie (ordered by ID) and ranking is weak where nobody fits (JD04 Spearman 0.37). Two process bugs were caught and fixed: a caregiving break counted as work (§7), and quota failures scored as empty resumes (#14).

10. What is misleading about my headline number

- **NDCG@5 = 0.98 and MRR = 1.0 sit near the ceiling by construction.** Pools of 20 have few good candidates. Spearman 0.65 is the honest ranking number, and the zero-shot LLM gets 0.60.
- **The labeller isn't independent.** Claude wrote the scoring design, the rubric and the labels. The rubric shares the system's conventions, and $\kappa = 0.98$ with an LLM second rater shows consistency, not truth. There are no human recruiter labels.
- **The data is synthetic and templated.** Extraction F1 = 1.0, YOE MAE = 0, zero run-to-run variance and exactly-zero fairness deltas are all easier on 75-word template resumes than on real ones. Evidence weighting, date parsing and projects are exercised mainly by unit tests.
- **Small n.** 7 test JDs and 10 auto-advances. Precision 10/10 has a Wilson 95% lower bound of about 0.72.
- **Quota-limited evidence.** The blind fairness mode is missing and fairness used 30 resumes, not 40. Judge agreement is against AI-made reference ratings (same family as the judge), not yet against a human.
- **Test exposure.** JD extraction prompts were iterated while looking at all 10 JD extractions (not at labels or metrics). The post-hoc ablations and both v1.1 changes were made after seeing test errors, so v1.1's test numbers are optimistic and are kept out of the headline.

11. Next week

Human recruiter labels (2 raters, adjudicated). A real resume corpus, including genuine career switchers, to decide whether `CAREER_SWITCH` should ever block again, and a fresh test set to confirm v1.1. Templated explanations for codes and gaps. Measuring the judge against the author's own ratings of the 40 explanations. Section-scoped skill extraction against trigger-free injection. Finishing blind-mode fairness. Feeding recruiter requirement edits and stage overrides back as labels. A calibrated probability of "should advance".